Computer Network Course Practice

Design and Implementation of a Lightweight Email Client

Report

Student ID: _____

Name: _____

Date: _____

School of Computer Science and Technology
Beijing Institute of Technology

2026 Fall

Contents

1 Requirements **3**

1.1 Basic requirements and what was implemented 3

1.2 Bonus features 3

2 Analysis **4**

2.1 SMTP: a command–reply protocol with numeric codes 4

2.2 POP3: status lines and dot-terminated lists 4

2.3 TCP is a byte stream 4

2.4 Message format 5

2.5 Local storage 5

2.6 Failure cases 5

2.7 Choice of test server 5

3 Design **5**

3.1 Architecture 5

3.2 Main data flow: sending and receiving 6

3.3 Design decisions 7

3.4 Error handling design 7

4 Development and Implementation **8**

4.1 Development environment 8

4.2 Project structure 8

4.3 SMTP client 8

4.4 POP3 client 10

4.5 TCP stream handling 11

4.6 Message building and parsing 11

4.7 Local mailbox 12

4.8 Shared use cases 12

4.9 Logging 12

5 System Deployment, Startup, and Use **13**

5.1 Requirements 13

5.2 Start the test server 13

5.3 Configure 13

5.4 Build and run the CLI 13

5.5 Run the web interface (bonus) 14

6 System Test **14**

6.1 Required tests (spec §5) 14
6.2 Error cases 15
6.3 Automated tests 15
6.4 Web interface 16

7 Performance **16**

8 Summary or Conclusions **17**

9 References **18**

10 Comments **19**

1 Requirements

The project is a lightweight email client that talks to mail servers over plain TCP sockets. It supports one account on one test server, sends plain-text mail with a subset of SMTP, receives mail with a subset of POP3, shows an Inbox, and keeps downloaded messages in a local file so they are still there after a restart. The protocol code is written directly against the socket; no library that hides SMTP or POP3 is used.

1.1 Basic requirements and what was implemented

Table 1 lists every required function from the project description (§2) and where it is implemented. All of them are complete.

Table 1: Required functions (project description §2) and their implementation.

	Requirement	Where	
	One account and one test server; address, SMTP port, POP3 port, username and other settings in a configuration file; clear errors when the server is unreachable or login fails	`config.json`, `pkg/config`	✓
	Send plain text to one or more recipients with `EHLO`/`HELO`, `MAIL FROM`, `RCPT TO`, `DATA`, `QUIT`; user enters recipients, subject and body	`pkg/smtp`	✓
	Receive with `USER`/`PASS`, `STAT`/`LIST`, `RETR`, `QUIT`; Inbox shows number, sender, subject, date; user selects and reads one message	`pkg/pop3`, `pkg/message`	✓
	Save downloaded messages locally and view them again after a restart	`pkg/mailbox` (JSON)	✓
	User interface: the five-item menu (Check Inbox, Read Message, Compose and Send, View Local Messages, Exit)	`cmd/cli`	✓
	Survive connection failure, timeout, wrong password, unexpected response, interrupted transfer and invalid input without crashing	`pkg/mailerr`	✓
	Log important events; never write a password to the log	`pkg/logger`, `pkg/pop3`	✓

1.2 Bonus features

These go beyond the basic requirements and are kept separate from them. The command-line client does not depend on any of them.

- **Graphical interface** (listed bonus). A web interface built with Vite and Tailwind CSS. It talks to a small JSON API in the Go program (`cmd/serve`), which uses exactly the same SMTP, POP3 and mailbox code as the CLI. It has Inbox, Read, Compose and Saved screens, a live SMTP/POP3 status light, and works at phone width.
- **Multiple accounts** (listed bonus). In the web interface any account on the mail server can sign in. The password is checked with a real POP3 `USER`/`PASS` exchange, and each account gets its own saved mailbox under `data/accounts/<email>/`.
- **Safe concurrency** (part of the "improved concurrency" bonus). The HTTP server handles each request on its own goroutine. The mailbox is guarded by a read/write lock, and POP3 sessions for one account are serialised by a mutex, while sending is not blocked by a running download. The whole test suite passes under Go's race detector.
- **Small extras**: a profile picture per account, and a light/dark theme toggle (☀ / ☾) carried over from an earlier project of mine.

Not implemented: TLS/STARTTLS, MIME attachments, IMAP, HTML mail, background checking.

2 Analysis

2.1 SMTP: a command–reply protocol with numeric codes

SMTP (RFC 5321 [1]) is a text protocol on TCP. The client sends one command per line, ended with `\r\n`, and the server answers each command with a reply whose first three characters are a status code. The first digit carries the meaning: `2xx` success, `3xx` "go on, send more" (only `354` after `DATA`), `4xx` temporary failure, `5xx` permanent failure. The client therefore must check each code before it sends the next command; for example, it must not send the message body unless `DATA` was answered with `354`.

The codes this client expects are: `220` greeting, `250` for `EHLO`, `MAIL FROM`, `RCPT TO` and the end of data, `354` for `DATA`, and `221` for `QUIT`. Two details of the reply format matter:

- **Multi-line replies.** A reply can span several lines. `250-text` (code, hyphen) means more lines follow; `250 text` (code, space) is the last line. GreenMail answers `EHLO` with three lines, so reading "one line" would leave two lines unread and every later reply would be matched to the wrong command.
- **End of data and dot-stuffing.** After `354` the message is sent as lines and ended by a line holding only a dot (`\r\n.\r\n`). A body line that itself starts with a dot must be sent with an extra dot ("`..`"), and the receiver removes it again. Without this a body line containing just "`.`" would end the message early.

2.2 POP3: status lines and dot-terminated lists

POP3 (RFC 1939 [2]) has no numeric codes. Every reply starts with `+OK` or `-ERR`. Replies to `LIST` and `RETR` are multi-line: after the `+OK` line the server sends data lines and ends them with a single dot, with the same dot-stuffing rule as SMTP. A session goes through three states: *authorization* (`USER`, `PASS`), *transaction* (`STAT`, `LIST`, `RETR`) and *update*, which begins at `QUIT`. Two consequences shaped the design:

- Message numbers are only valid inside one session. Message 1 today may be another message tomorrow. So the local mailbox gives every saved message its own permanent ID instead of reusing the POP3 number.
- The server locks the maildrop while a session is open, so a second parallel login for the same account is refused. The client must not run two POP3 sessions for one account at the same time.

2.3 TCP is a byte stream

TCP delivers an ordered stream of bytes, not messages. One `write` by the server can arrive in several `read` calls, and one `read` can contain the end of one reply and the start of the next. This was measured in this project: a raw socket received a 104 KB `RETR` reply from GreenMail in **672 separate reads**, about 155 bytes each (Section 7). The protocol layer therefore must buffer the incoming bytes and cut them into lines at `\r\n`, and then decide from the line content whether a reply is finished: by the space after the code in SMTP, or by the lone dot in POP3 multi-line replies. Figure 6 shows this in the client.

2.4 Message format

The text after `DATA` and the text returned by `RETR` follow the Internet Message Format (RFC 5322 [3]): header lines such as `From:`, `To:`, `Subject:` and `Date:`, a blank line, then the body. For the Inbox the client needs sender, subject and date, which it reads from these headers. A subject with non-ASCII characters is written as an RFC 2047 encoded word [4], e.g. `=?utf-8?q?caf=C3=A9?=`, and has to be decoded for display.

2.5 Local storage

The spec allows text files, JSON or SQLite. A single JSON file was chosen: it needs no extra software, can be opened and checked by hand, and is fast enough for a personal mailbox of a few hundred messages. Two problems had to be handled: pressing "Check Inbox" twice must not save every message twice, and a crash while saving must not destroy the file.

2.6 Failure cases

Each failure appears at a different layer and must be told apart so the user gets a useful message:

	Failure	**How it shows up**
	Server unreachable	`connect` fails with "connection refused", or a DNS error
	Timeout	a socket read or write passes its deadline
	Wrong username/password	POP3 answers `USER` or `PASS` with `-ERR`
	Server refuses	SMTP `4xx/5xx` (e.g. `550` on `RCPT`), or another POP3 `-ERR`
	Unexpected response	a line that is not `NNN text` or `+OK/-ERR`
	Interrupted transfer	the socket closes before the terminating dot (EOF, reset)
	Invalid input	bad menu choice, non-numeric message number, malformed address

2.7 Choice of test server

The client was developed against **GreenMail** [6], an open-source test mail server that provides SMTP and POP3 in one Docker container. It was chosen because it is an independent, standard implementation, so the client is tested against a server I did not write; it checks passwords, which the wrong-password test needs; it runs on `localhost` with no TLS or OAuth, which the basic project leaves out; and it is started and reset with one command, which makes the tests repeatable. It listens on ports 3025 (SMTP) and 3110 (POP3) instead of 25 and 110, which need administrator rights.

3 Design

3.1 Architecture

The client is split into small Go packages with one job each (Figure 1). Two thin front ends, the CLI and the HTTP API for the web interface, sit on one shared core. The protocol code exists exactly once, so both front ends send and receive mail in the same way and the tests of the core cover both.

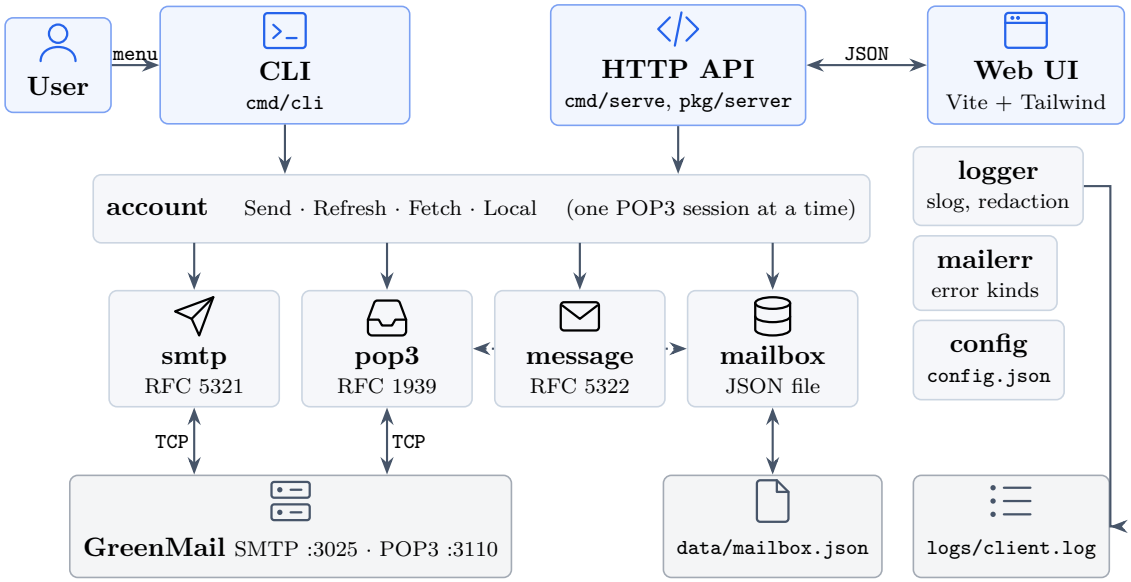


Figure 1: Module structure. Blue: front ends. Grey: the shared core and the outside world. `logger`, `mailerr` and `config` are used by every package. Only `smtp` and `pop3` open sockets; `message` only builds and parses text.

Table 2: Module responsibilities.

Package	Responsibility
`config`	Load `config.json`, apply defaults, reject unknown keys, report *all* invalid fields at once. The password may come from `EMAIL_PASSWORD` instead of the file.
`logger`	Structured log (`log/slog`) to `logs/client.log`; any attribute named `password`, `pass` or `secret` is replaced by `[REDACTED]`.
`smtp`	One SMTP session over a raw socket: greeting, `EHLO` (fallback `HELO`), `MAIL FROM`, `RCPT TO` per recipient, `DATA`, `QUIT`. Checks every reply code.
`pop3`	POP3 over a raw socket: `USER`/`PASS`, `STAT`, `LIST`, `RETR`, `QUIT`. Parses `+OK`/`-ERR`; masks `PASS` in the log.
`message`	Build the outgoing text (headers + body); parse incoming text into From, To, Subject, Date, Body; validate recipient addresses.
`mailbox`	Local store: add with duplicate check (SHA-256), permanent local IDs, atomic save to JSON.
`account`	The four use cases (send, refresh inbox, fetch one, list local) shared by both front ends.
`mailerr`	Turn any error into one of eight kinds and a plain-language sentence.
`server`	HTTP JSON handlers, sign-in sessions, static files for the web UI.

3.2 Main data flow: sending and receiving

Figure 2 follows one message through the whole system. On the way out the client builds the message text and hands it to the SMTP session. GreenMail stores it in the recipient's maildrop. On the way in a POP3 session lists and retrieves it, the text is parsed into fields, and the mailbox saves it with a new local ID. The Inbox is drawn from that saved copy.

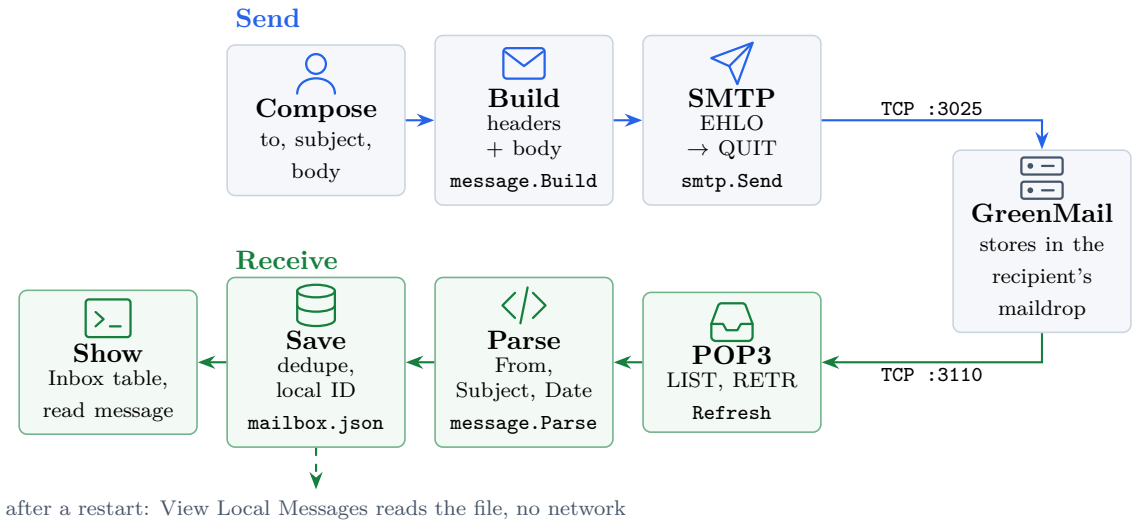


Figure 2: The path of one message from Compose to the Inbox.

3.3 Design decisions

- **One session per action.** Every send and every inbox check opens a new connection and closes it with `QUIT`. There is no long-lived connection to keep alive or to recover after a network error.
- **A fresh deadline before every step.** The socket deadline is reset before each command, so a slow but working server is fine while a server that stops answering fails after `timeout_seconds` instead of hanging forever.
- **Local ID separate from POP3 number.** The Inbox and "Read Message" use the server's number for the current session; the saved copy keeps a local ID that never changes and is never reused.
- **Errors are values, handled in one place.** Every function returns an error wrapped with the step that failed (e.g. `smtp: RCPT TO bob@x: 550 ...`). The CLI menu loop and the HTTP handler each pass it to `mailerr.Explain` and carry on, so no error can end the program.

3.4 Error handling design

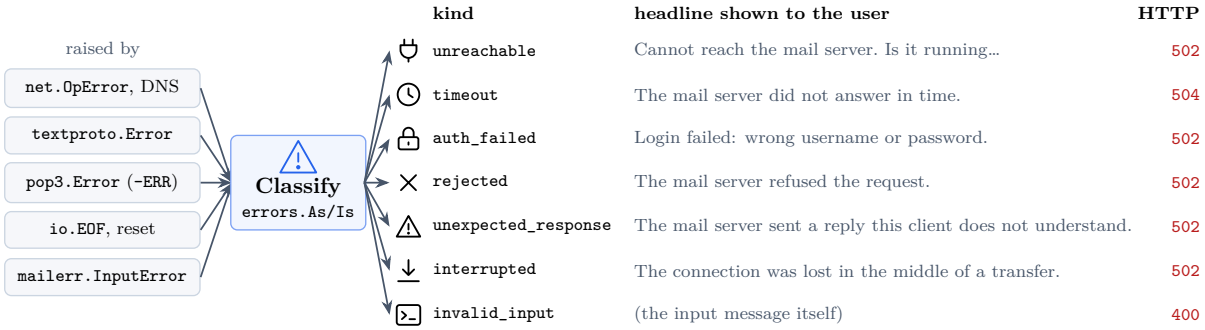


Figure 3: Every error, whatever layer it came from, is sorted into one kind. The CLI prints the headline and a `detail:` line; the API returns `{error, kind, detail}` with the status code on the right.

The order of the checks matters: a wrong password is also a POP3 `-ERR`, so `auth_failed` (an `-ERR` to `USER` or `PASS`) is tested before the general `rejected` case.

4 Development and Implementation

4.1 Development environment

Operating system	macOS 15.7 on Apple Silicon (arm64)
Language	Go 1.26.7 (client, API, tests)
Standard library	`net`, `net/textproto`, `net/mail`, `mime`, `encoding/json`, `log/slog`, `crypto/sha256`, `net/http`, `text/tabwriter`
Third-party Go	none
Web interface	JavaScript (no framework); Node.js 25.7, Vite 8 and Tailwind CSS 4 as build tools only
Test server	GreenMail 2.1.14 in Docker 29.8
Storage	one JSON file per account; no database

Why `net/textproto` is allowed. The spec forbids a high-level library that hides SMTP. Go has one, `net/smtp`, and it is *not* used. `net/textproto` only does line framing: read a line up to `\r\n`, write a line, read a numeric reply including its continuation lines, and read/write dot-terminated blocks. It knows no SMTP or POP3 command. Every command, its order, and the check of every reply are written in this project.

4.2 Project structure

```
NetworkLab/
|-- Client/                Go module "email-client"
|   |-- config.json        server, ports, username, timeout, paths
|   |-- cmd/cli/           terminal menu
|   |-- cmd/serve/         HTTP API (+ serves the built web UI)
|   `-- pkg/  account  config  logger  smtp  pop3  message  mailbox  mailerr  server
|-- Tests/                 Go module "email-client-tests": unit, integration, perf tests
|                          and section5.sh, which regenerates all evidence in Results/
|-- Results/               transcripts, log, mailbox.json, timing, screenshots
`-- WebUI/                 Vite + Tailwind web interface
```

4.3 SMTP client

All SMTP commands go through one helper. It resets the deadline, logs the line, sends it with `\r\n` and then reads a *complete* reply. `ReadResponse` keeps reading while lines have the form `250-...`, so a multi-line reply split over several TCP reads is still read as one reply, and it fails if the code is not the expected one.

```
func (c *Client) cmd(expect int, format string, args ...any) error {
        line := fmt.Sprintf(format, args...)
        c.extend()                          // SetDeadline(now + timeout)
        c.log.Debug("C: " + line)
        if err := c.tp.PrintfLine("%s", line); err != nil {
                return err
        }
        code, msg, err := c.tp.ReadResponse(expect)  // whole reply, all lines
```

```
        c.log.Debug("S: "+msg, "code", code)
        return err
}
```

The body is written through a `DotWriter`. It turns every `\n` into `\r\n`, doubles a leading dot on any line, and writes the closing `.\r\n` when it is closed. Only then does the client wait for 250:

```
func (c *Client) Data(msg []byte) error {
        if err := c.cmd(354, "DATA"); err != nil {
                return fmt.Errorf("smtp: DATA: %w", err)
        }
        c.extend()
        w := c.tp.DotWriter()
        if _, err := w.Write(msg); err != nil { ... }
        if err := w.Close(); err != nil { ... }        // sends "\r\n.\r\n"
        code, reply, err := c.tp.ReadResponse(250)
        ...
}
```

`Send` runs the session in the order of Figure 4: connect and read `220`, `EHLO` (and `HELO` if the server answers `5xx` to `EHLO`), `MAIL FROM`, one `RCPT TO` per recipient, `DATA`, `QUIT`. `QUIT` expects `221`, not `250` (RFC 5321 §4.2.2). A `defer c.Close()` releases the socket on every error path.

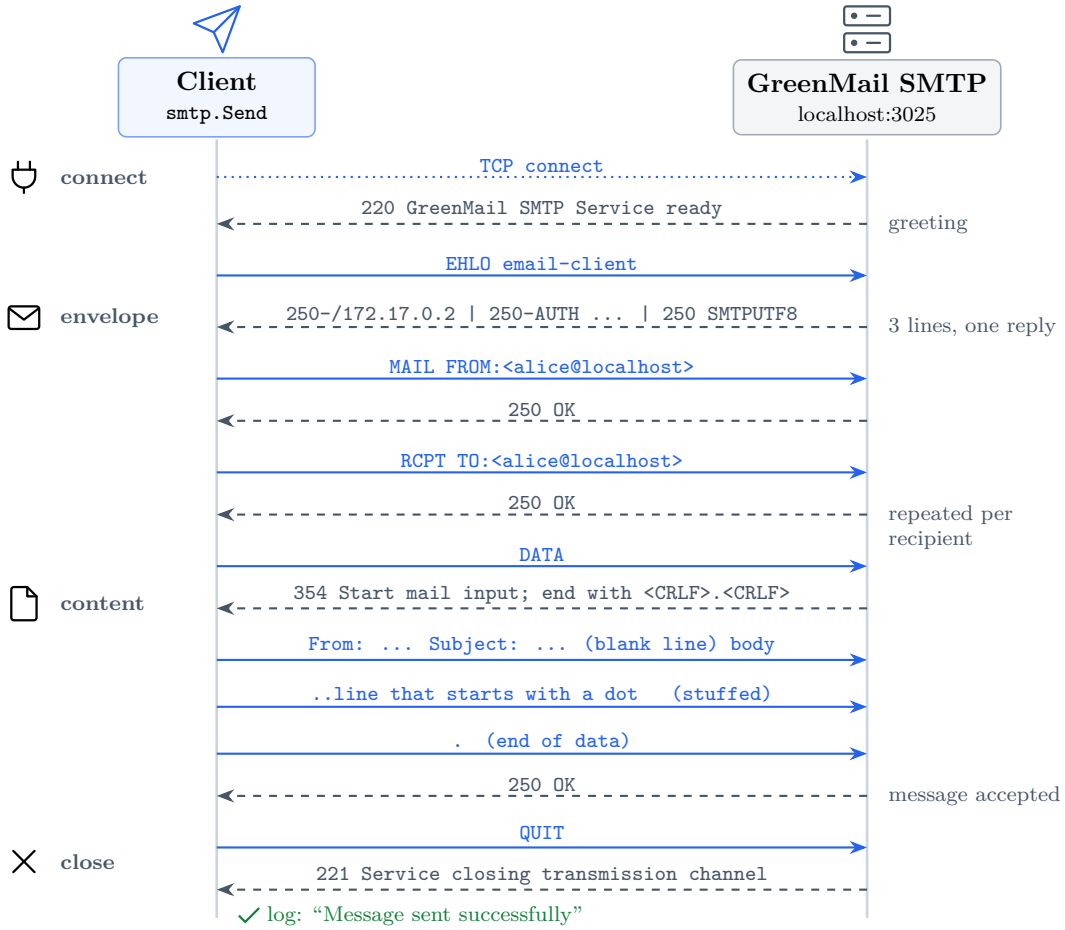


Figure 4: The SMTP send session as captured in `Results/client.log`. Solid blue arrows are client commands, dashed grey arrows server replies. Each reply code is checked before the next command is sent; any other code ends the session with an error that names the step.

4.4 POP3 client

POP3 replies are parsed by hand, since they have no numeric code. One status line is read, split at the first space, and sorted into three outcomes. Anything other than `+OK` or `-ERR` means the peer is not speaking POP3 and is reported as an unexpected response:

```
func (c *Client) reply(cmd string) (string, error) {
        c.extend()
        line, err := c.tp.ReadLine()          // buffered: waits for a full "\r\n" line
        if err != nil {
                return "", fmt.Errorf("pop3: %s: %w", cmd, err)
        }
        status, rest, _ := strings.Cut(line, " ")
        switch status {
        case "+OK":  return rest, nil
        case "-ERR": return "", &Error{Cmd: cmd, Msg: rest}
        default:
                return "", fmt.Errorf("pop3: %s: %w %q", cmd, ErrUnexpectedResponse, line)
        }
}
```

The command helper masks the password before the line reaches the log; the real line still goes to the socket:

```
        name, _, _ := strings.Cut(line, " ")
        shown := line
        if name == "PASS" {
                shown = "PASS ****"
        }
```

`LIST` reads the data lines with `ReadDotLines`; `RETR` reads the body with a `DotReader`, which stops at the lone dot, removes dot-stuffing and converts `\r\n` to `\n`. If the socket closes before the dot arrives, `ReadAll` returns `unexpected EOF`, reported as “transfer interrupted”.

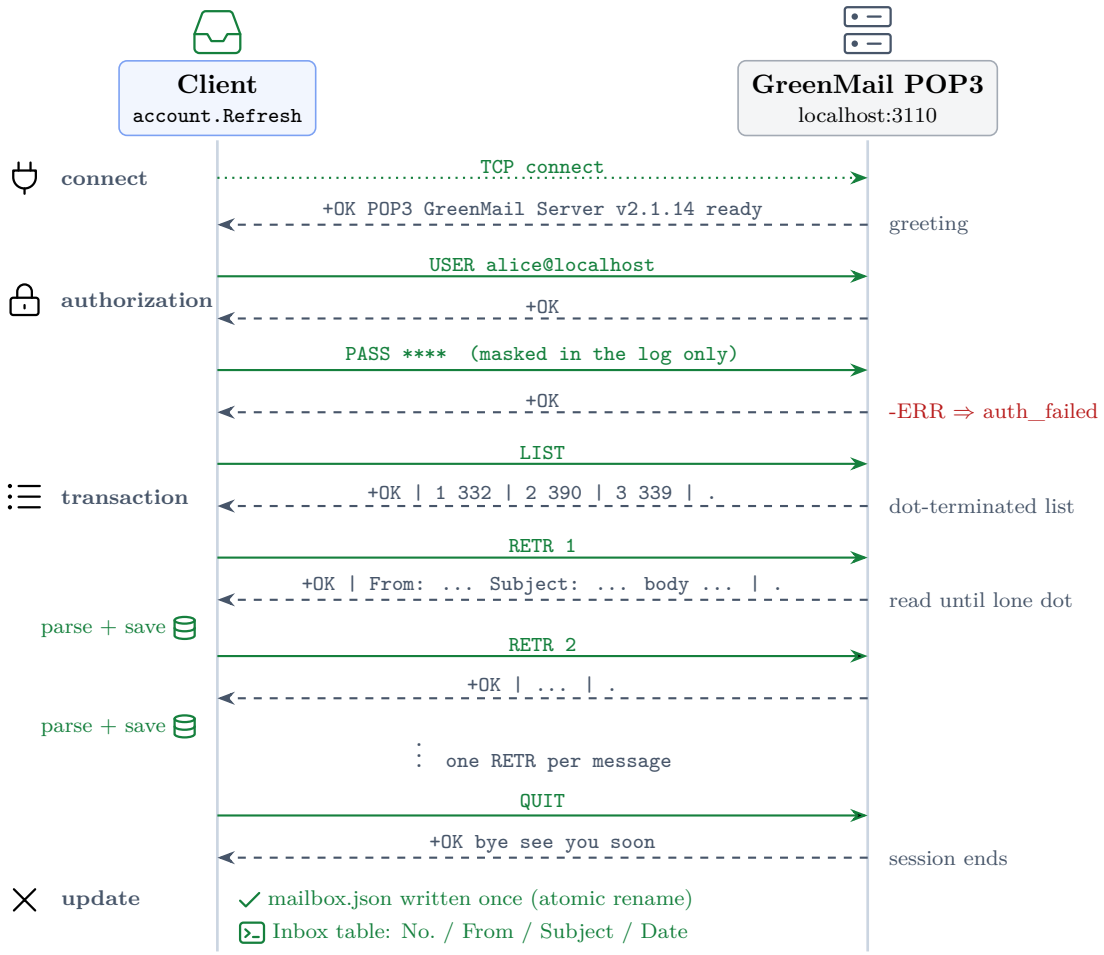


Figure 5: The POP3 receive session (Check Inbox), as captured in `Results/client.log`. "Read Message" is the same session with a single `RETR n` and no `LIST`.

4.5 TCP stream handling

Figure 6 shows how the client turns a stream of TCP segments into protocol replies. Neither the SMTP nor the POP3 code ever calls `read` on the socket directly. A buffered reader collects bytes until a full line is present, and the reply logic decides from the line content whether more lines belong to the same reply.

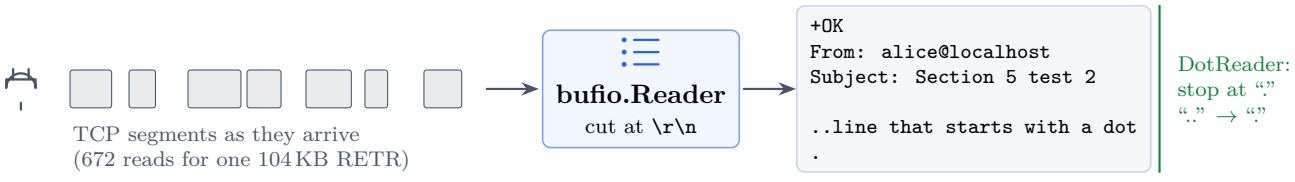


Figure 6: Bytes are buffered and cut into lines; a reply is complete only when its last line arrives (`NNN text` in SMTP, a lone dot for POP3 multi-line replies), however the bytes were split on the wire.

4.6 Message building and parsing

`message.Build` writes `From`, `To`, `Subject`, `Date` (RFC 1123Z), `MIME-Version`, `Content-Type: text/plain; charset=utf-8` and `Content-Transfer-Encoding: 8bit`, a blank line and the body. The subject is Q-encoded when it contains non-ASCII or control characters; besides

correct display this stops a subject such as `"hi\nBcc: x"` from injecting a second header. `message.Parse` uses `net/mail` (text parsing only, no networking) and decodes encoded words for display. A missing or malformed `Date` is shown as "unknown" instead of failing. `ParseRecipients` splits the user's "To" field on commas and accepts both `bob@x` and `Bob <bob@x>`.

4.7 Local mailbox

Each saved entry holds a permanent local `id`, the `pop3_num` it had when it was downloaded, a SHA-256 `hash` of the raw text, the parsed fields, and the raw text itself. `Add` skips a message whose hash is already present, so checking the inbox twice saves nothing twice. `Save` writes `mailbox.json.tmp` and renames it over the real file. A rename is atomic on one filesystem, so a crash leaves either the old file or the new one, never half of each. A read/write lock makes the mailbox safe for the HTTP server's concurrent requests.

4.8 Shared use cases

Every POP3 action has the same shape, so it is written once:

```
func (a *Account) session(fn func(*pop3.Client) error) error {
        a.popMu.Lock()                  // one POP3 session per account (maildrop lock)
        defer a.popMu.Unlock()
        c, err := pop3.Dial(a.cfg.POP3Addr(), a.cfg.Timeout(), a.log)
        if err != nil {
                return err
        }
        defer c.Close()                 // released on every error path
        if err := c.Auth(a.cfg.Username, a.cfg.Password); err != nil {
                return err
        }
        if err := fn(c); err != nil {  // LIST + RETR all, or one RETR
                return err
        }
        return c.Quit()
}
```

`Refresh` saves what it has already downloaded even if a later `RETR` fails, so a dropped connection does not throw away messages already received.

4.9 Logging

The log uses `log/slog`. At the default level it records events only; with `-debug` it also records every protocol line as `C:`/`S:`. Passwords are protected twice: the logger replaces any attribute named `password`, `pass` or `secret`, and the POP3 client masks the `PASS` command itself. The log file is created with permission `0600`.

```
level=INFO  msg="Connected to SMTP server" addr=localhost:3025
level=INFO  msg="Message sent successfully" to=[alice@localhost]
level=INFO  msg="Connected to POP3 server" addr=localhost:3110
level=DEBUG msg="C: PASS ****"
level=INFO  msg="Retrieved message 2" bytes=390
level=ERROR msg="POP3 authentication failed" user=alice@localhost err="pop3: PASS: ..."
```

5 System Deployment, Startup, and Use

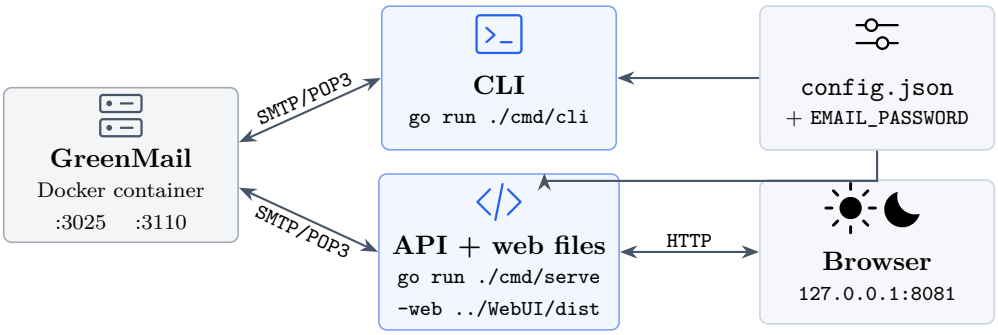


Figure 7: The two ways to run the client. Both read the same `config.json` and talk to the same GreenMail server. The browser page has the light/dark toggle.

5.1 Requirements

Go 1.26.7 or newer (the version in `go.mod`) and Docker. Node.js is needed only to rebuild the web interface; the CLI needs nothing besides Go.

5.2 Start the test server

```
docker run -d --name greenmail -p 3025:3025 -p 3110:3110 \
  -e GREENMAIL_OPTS="-Dgreenmail.setup.test.all -Dgreenmail.hostname=0.0.0.0 \
    -Dgreenmail.users=alice:alice@localhost,bob:bob@localhost \
    -Dgreenmail.users.login=email -Dgreenmail.verbose" \
  greenmail/standalone
```

`GREENMAIL_OPTS` creates two test accounts, `alice@localhost` and `bob@localhost`, which log in with their full email address. A wrong password is rejected, which the error tests rely on.

5.3 Configure

`Client/config.json` (no real passwords are used anywhere in this project):

```
{
  "server": "localhost", "smtp_port": 3025, "pop3_port": 3110,
  "username": "alice@localhost", "from": "alice@localhost",
  "timeout_seconds": 10,
  "mailbox_path": "data/mailbox.json", "log_path": "logs/client.log"
}
```

The password can be put in the file as `"password"`, or, better, given in the environment as `EMAIL_PASSWORD` so it is never stored. A misspelled key (`"smpt_port"`) or an invalid value is reported at startup, before any connection.

5.4 Build and run the CLI

```
cd Client
EMAIL_PASSWORD=... go run ./cmd/cli            # or: go build -o emailcli ./cmd/cli
EMAIL_PASSWORD=... go run ./cmd/cli -debug     # also log every C:/S: protocol line
```

The menu offers the five functions of the spec. **1 Check Inbox** downloads the inbox, saves new messages and prints a table. **2 Read Message** asks for a number from that table and retrieves the message. **3 Compose and Send Email** asks for recipients (comma-separated), subject and body; the body ends with a line holding only a dot. **4 View Local Messages** lists the saved copies without any network access and opens one by local ID. **5 Exit**.

5.5 Run the web interface (bonus)

```
cd WebUI  && npm install && npm run build
cd Client && go run ./cmd/serve -web ../WebUI/dist     # open http://127.0.0.1:8081
```

The server listens only on `127.0.0.1` by default, because the session cookie travels over plain HTTP. Sign in with a GreenMail account (e.g. `bob@localhost`); the sidebar leads to Inbox, Compose and Saved, and the status bar shows whether SMTP and POP3 are up.

6 System Test

All results were produced on 9 October 2026 by one script, `Tests/section5.sh --reset`, which restarts GreenMail with an empty inbox and writes every transcript, log and table into `Results/`. Running it again reproduces them.

6.1 Required tests (spec §5)

Table 3: The five required tests and their results.

#	Requirement	Result	Evidence
1	Connect to SMTP and POP3	Pass. SMTP `220` greeting, `250` to `EHLO`, `221` to `QUIT`. POP3 `+OK` greeting, login accepted, `STAT` → `+OK 0 0`.	`01-connection.txt`
2	Send at least three plain-text messages	Pass. Three sends from the CLI (53, 11 and 6 ms); the second to two recipients. A fourth from the web UI.	`02-send-...txt`
3	Retrieve and display	Pass. Inbox lists all three with number, sender, subject, date. Messages 2 and 3 open in full; the body line "`.line that starts with a dot`" arrives intact.	`02-send-...txt`
4	Save and reopen after restart	Pass. A new CLI process lists the three saved messages and opens ID 3 without contacting the server.	`03-reopen-...txt`
5	At least three error cases	Pass, four cases (Table 4). No crash; every run exits with code 0.	`04-error-cases.txt`
–	Password never logged	Pass. 0 lines contain the real or the wrong password; 4 lines show `PASS ****`.	`05-log-check.txt`

Excerpt of test 2 and 3 (the typed input is listed before the output in the transcript):

```
> To (comma-separated): Subject: Body (end with a line containing only "."):
```

```
Sent to alice@localhost, bob@localhost in 11 ms
...
3 message(s) on server, 3 new saved locally

No.  From             Subject           Date
1    alice@localhost  Section 5 test 1  2026-10-09 16:14
2    alice@localhost  Section 5 test 2  2026-10-09 16:14
3    alice@localhost  Section 5 test 3  2026-10-09 16:14

> Message number (from Check Inbox):
From:    alice@localhost
To:      alice@localhost, bob@localhost
Subject: Section 5 test 2
Date:    2026-10-09 16:14

Second message, sent to two recipients.
.line that starts with a dot
```

6.2 Error cases

Table 4: Error cases run against the CLI.

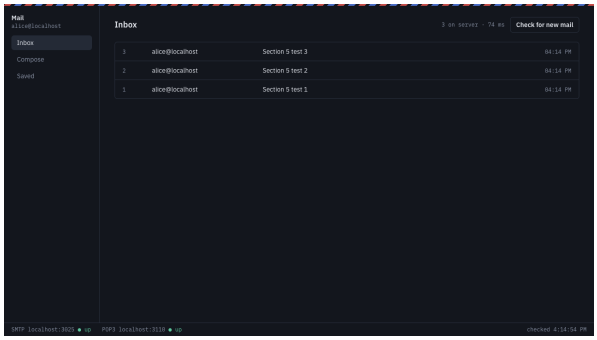
	Case	How it was caused	What the user sees
	Wrong password	`EMAIL_PASSWORD=wrong-password`	`Login failed: wrong username or password.` with `detail: ... -ERR ... Invalid password`
	Server unavailable	both ports set to 1, where nothing listens	`Cannot reach the mail server. Is it running, and are the address and port right?` for both Check Inbox and Send
	Timeout	a listener that accepts and never sends a greeting; `timeout_seconds: 2`	`The mail server did not answer in time.` after about 2 s
	Invalid input	menu `9`, number `abc`, recipient `not-an-address`, saved ID `99`	one specific message for each, then the menu again

Two more failure kinds are covered by automated tests with fake servers: a connection cut in the middle of `RETR` ("The connection was lost in the middle of a transfer") and a server that does not speak the protocol ("The mail server sent a reply this client does not understand").

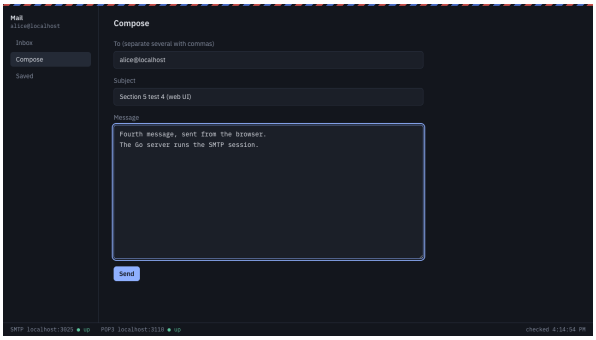
6.3 Automated tests

The `Tests/` module holds unit tests for every package (with small fake SMTP/POP3 servers on `127.0.0.1` for the protocol and error cases), CLI tests that feed the menu through a pipe, HTTP API tests, and integration tests against the real GreenMail. All **51 tests pass** with `go test -race ./...`, which also checks the concurrent code for data races. Examples of what they check: dot-stuffing in both directions; a multi-line `EHLO` reply; fallback from `EHLO` to `HELO`; a `550` on one recipient; a reply split across many small TCP writes; the log never containing the password; duplicate detection and the atomic save of the mailbox.

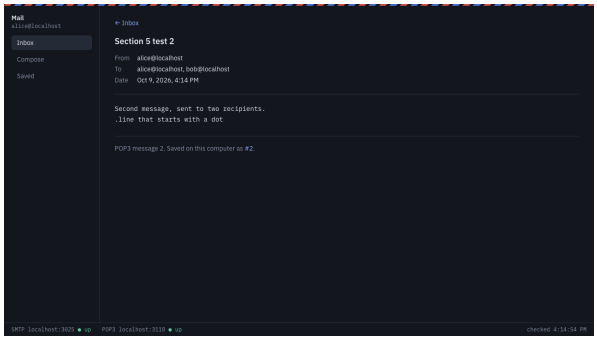
6.4 Web interface



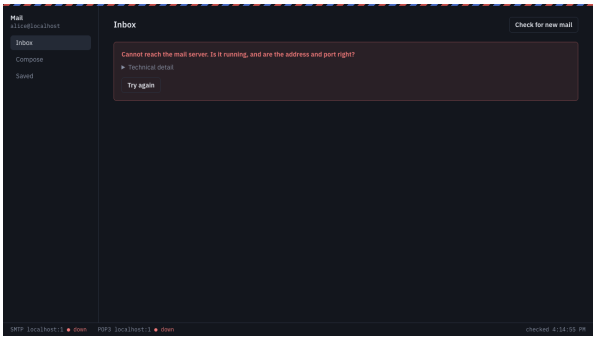
(a) Inbox



(b) Compose



(c) Read message



(d) Error: server down

Figure 8: The web interface running the same tests (all screenshots are in `Results/screenshots/`).

7 Performance

`Tests/perf` measures whole protocol sessions (TCP connect, commands, `QUIT`) against GreenMail on the same machine. Sends and single retrievals ran 10 times per size; the full inbox check ran 5 times. The bodies are realistic 72-character text lines. The test uses bob's account so the demo inbox is not filled.

Table 5: Session times in milliseconds (macOS, Apple Silicon, Go 1.26.7, GreenMail 2.1.14 in Docker, localhost).

Operation	Size	Min	Median	Mean	Max
Send (SMTP session)	1 KB	4.9	7.8	11.9	28.3
	10 KB	5.8	7.3	8.2	18.9
	100 KB	18.2	21.4	21.1	28.2
Retrieve one (RETR)	1 KB	2.6	3.0	3.1	4.1
	10 KB	8.2	8.7	8.8	10.6
	100 KB	63.7	66.1	66.1	68.3
Check inbox (LIST + RETR all 31)	mixed	721.5	734.8	748.5	808.0

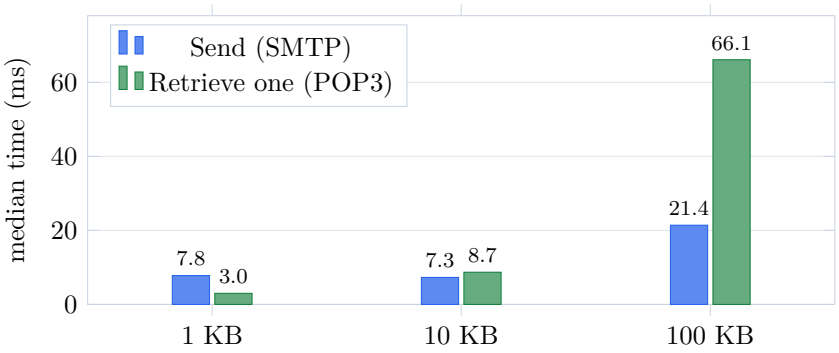


Figure 9: Median send and retrieve times by message size.

Discussion.

- **Small messages cost the same.** Sending 1 KB and 10 KB both take about 7–8 ms. That time is the TCP connect plus five command round trips, not the data.
- **The first session is slower.** The first send in the CLI run took 53 ms and the slowest timed send 28 ms, against 5–11 ms for later ones: the first connection warms up GreenMail's Java code and Docker's port forwarding. The median ignores this outlier, the mean does not, which is why the 1 KB mean is well above its median.
- **Retrieving 100 KB takes about three times as long as sending it.** The cause is on the server side. A raw Python socket, with no Go code involved, received a 104 KB `RETR` reply in 672 `recv()` calls of about 155 bytes: GreenMail writes roughly one line per TCP segment, and every segment crosses Docker's port forwarding. Sending is faster because the client's buffered writer sends large segments. The client reassembled all 672 pieces correctly, which is the byte-stream property of Section 2.3 in practice.
- **Check Inbox grows with the mailbox.** 31 messages (10 of them 100 KB) take about 0.73 s, because the client retrieves every message to show its sender, subject and date. POP3 `TOP n 0` (headers only) would make this much cheaper; it is not in the required command set and was not implemented.

8 Summary or Conclusions

Completed. All basic requirements are implemented and tested: configuration with clear startup errors, SMTP sending to one or more recipients, POP3 receiving with an Inbox and message view, a local JSON mailbox that survives restarts, handling of every failure kind the spec lists, and an event log that never contains a password. As bonus features there is a web interface, sign-in with multiple accounts each with its own mailbox, and concurrency-safe shared state verified by the race detector. The five required tests pass, as do 51 automated tests.

Main technical problems and their solutions.

- *Multi-line SMTP replies.* GreenMail answers `EHLO` with three lines. Reading only one line per command would match every later reply to the wrong command. The client reads the complete reply (all `250-` continuation lines) before sending the next command.
- *Partial reads.* A reply can arrive in hundreds of pieces (672 for one 100 KB message). All reads go through a buffered line reader, and completeness is decided by the protocol's own end markers, never by the size of a read.

- *The end-of-data dot.* A body line holding only "." would end a message early. Dot-stuffing is applied on send and removed on receive, and the test run includes a body line that starts with a dot.
- *POP3 numbers change between sessions.* Saved messages get their own permanent ID, and a SHA-256 hash stops the same message being saved twice.
- *Passwords in logs.* The `PASS` line is masked at the protocol layer and the logger redacts password attributes; a test checks the log file.
- *A server that stops answering.* A blocking read would wait forever. A deadline is set before every step, so the client gives up after `timeout_seconds`.
- *Parallel POP3 logins.* The server locks the maildrop during a session, so two browser tabs refreshing at once would collide. POP3 sessions for one account are serialised by a mutex.

What I learned. *Your own reflection: e.g. what was harder than expected, what reading the RFCs taught you (QUIT is answered with 221, not 250), and what testing against an independent server showed.*

Limitations and future work.

- No TLS/STARTTLS and no SMTP AUTH; the client is meant for a local test server.
- Check Inbox downloads every message. `TOP` or `UIDL` would let it fetch headers only, or only new messages.
- Messages are never deleted from the server (`DELE` is not used).
- Plain text only: no attachments or HTML mail.
- The web session cookie travels over plain HTTP, so the server listens on localhost only.

9 References

[1] J. Klensin, "Simple Mail Transfer Protocol," RFC 5321, IETF, Oct. 2008. `https://www.rfc-editor.org/rfc/rfc5321`

[2] J. Myers and M. Rose, "Post Office Protocol – Version 3," RFC 1939, IETF, May 1996. `https://www.rfc-editor.org/rfc/rfc1939`

[3] P. Resnick, "Internet Message Format," RFC 5322, IETF, Oct. 2008. `https://www.rfc-editor.org/rfc/rfc5322`

[4] K. Moore, "MIME Part Three: Message Header Extensions for Non-ASCII Text," RFC 2047, IETF, Nov. 1996. `https://www.rfc-editor.org/rfc/rfc2047`

[5] The Go Authors, Go standard library documentation: `net`, `net/textproto`, `net/mail`, `mime`, `log/slog`, `encoding/json`, `net/http`. `https://pkg.go.dev/std`

[6] GreenMail, "GreenMail – Test suite for email," version 2.1.14. `https://greenmail-mail-test.github.io/greenmail/`

[7] Vite, "Vite documentation." `https://vite.dev`

[8] Tailwind Labs, "Tailwind CSS documentation," v4. `https://tailwindcss.com/docs`

[9] Z. Liu, "Design and Implementation of a Lightweight Email Client," project description, Computer Network Course Practice, Beijing Institute of Technology, 2026.

10 Comments

Optional: comments or suggestions about the course, project, or teaching materials.

Submission note: Submit the complete source code, README, this report, representative screenshots/terminal output, test results/logs, and a short description of any bonus features. All work is individual; be prepared to explain the submitted code and protocol behavior during the presentation or demonstration.